

CV Matcher: Explainable CV–Job Matching System (ECMS)

Buse Erkiraz and Niyousha Mojoudi and Ertugrul Taparci and Ipek Uzun

buse.erkiraz@stud.uni-heidelberg.de

niyousha.mojoudi@stud.uni-heidelberg.de

ertugrul.taparci@stud.uni-heidelberg.de

ipek.uzun@stud.uni-heidelberg.de

Abstract

We present ECMS, an Explainable CV–Job Matching System that evaluates how well a candidate CV matches a job description using three complementary methods: dense embedding similarity (2a), structured LLM extraction with rule-based scoring and a gap explainer (2b), and direct LLM-as-judge evaluation (2c). To support evaluation, we construct a 30-pair gold-standard dataset spanning four job families and three fit levels, annotated by a human recruiter on a 1–5 scale. Comparing all three methods against human labels shows that accuracy does not simply scale with cost: Method 2c achieves the highest Spearman correlation with human judgment ($\rho = 0.79$, shortlisting accuracy 76.7%) at moderate cost, Method 2a offers instant, zero-cost candidate filtering with a moderate correlation of its own ($\rho = 0.52$, 66.7%), and Method 2b’s structured scoring is the most expensive of the three yet the least correlated with human judgment ($\rho = 0.18$, 63.3%), though it still provides full auditability via a structured diff and natural-language gap explanation. Bias robustness tests across 48 formatting variants and 108 demographic name variants show that name-based bias is negligible in embedding and rule-based methods but merits attention in LLM-based approaches.

1 Introduction

Automated CV screening is now standard in large-scale recruitment, yet the dominant approach which is keyword-matching Applicant Tracking Systems (ATS), is widely criticised for filtering out qualified candidates whose experience does not literally mirror job-description phrasing (Raghavan et al., 2020). Candidates with transferable skills, non-traditional career paths, or non-native English writing are disproportionately penalised, even when their underlying qualifications are strong.

Given a candidate CV and a job description, the task is to produce a score in $[0, 100]$ estimating the candidate’s fit for the role, accompanied by a struc-

tured explanation of the match. A practical system must handle three challenges simultaneously:

- **Semantic gap.** “Led a team that reduced time-to-fill by 30%” should match “management experience” without requiring literal overlap.
- **Constraint sensitivity.** A missing must-have requirement (required certification, minimum years of experience) should weigh more heavily than a missing nice-to-have preference.
- **Surface robustness.** Scores should not change when the CV is reformatted or the candidate’s name is swapped for one associated with a different demographic group.

We build and evaluate three methods that represent different positions on the speed–accuracy–explainability spectrum. **Method 2a** (Dense Embedding Similarity) encodes CV and JD text into dense vectors and computes a section-aware cosine similarity score; it requires no API calls and runs in under 100 ms. **Method 2b** (Structured Extraction + Rules) uses an LLM to parse both documents into a shared schema (skills, experience, education), scores each dimension deterministically, and feeds the resulting structured diff into a gap explainer that produces an auditable natural-language rationale. **Method 2c** (LLM-as-Judge) prompts an LLM directly with the raw CV and JD, grounded in a two-tier recruiting rubric (must-haves vs. nice-to-haves), returning a score and explanation in a single call.

All three methods share a common output schema, enabling direct comparison through a unified evaluation harness. We evaluate against a 30-pair gold-standard dataset with human annotations and conduct bias robustness tests across formatting layouts and demographic name variants. Our main findings are: (1) the LLM-as-judge method substantially outperforms both embedding similarity and structured extraction on correlation with

human judgment; (2) structured intermediate representations improve score interpretability but, in our implementation, do not recover the accuracy lost relative to embedding similarity despite costing far higher latency; (3) name-based demographic bias is negligible for non-LLM methods but present in the LLM judge.

2 Related Work

Automated CV screening has long relied on keyword-matching Applicant Tracking Systems (ATS), which are widely criticised for penalising qualified candidates whose phrasing does not literally mirror the job description (Raghavan et al., 2020). Earlier computational approaches shared the same surface-form brittleness, using TF-IDF cosine similarity and bag-of-words overlap to rank candidates. Subsequent work framed CV-JD matching as a representation learning task, training person-job fit models on implicit signals such as application outcomes and interview callbacks (Qin et al., 2018), but these approaches require large proprietary datasets and produce opaque scores with no structured rationale.

The introduction of contextualised representations (Devlin et al., 2019) and sentence-level encoders such as Sentence-BERT (Reimers and Gurevych, 2019) enabled semantic matching beyond keyword overlap, capturing paraphrases and implied experience that literal matching misses. Our Method 2a builds directly on this line of work; the moderate correlation we observe ($\rho = 0.517$) suggests that semantic proximity captures a real but incomplete share of recruiter judgment, falling short when recruiters weight hard constraints above topical relevance.

More recently, Zheng et al. (2023) established the LLM-as-judge paradigm, showing that a prompted language model can evaluate outputs with near-human reliability when grounded in a well-specified rubric. Our Method 2c adapts this approach to recruitment by grounding the judge in a two-tier must-have vs. nice-to-have rubric. To our knowledge, this is the first systematic evaluation of the LLM-as-judge paradigm against a human-annotated CV-JD gold standard.

Demographic bias in hiring has been documented since the audit study of Bertrand and Mullainathan (2004), who showed that identical CVs with White-associated names received significantly more callbacks than those with African-

American-associated names. In NLP, Bolukbasi et al. (2016) showed that word embeddings encode gender stereotypes, and De-Arteaga et al. (2019) demonstrated that occupation classifiers amplify demographic bias from surface features such as names. Our robustness tests extend this line of work to CV matching: by swapping candidate names for demographic proxies while holding content fixed, we find that embedding-based scoring is nearly immune ($\Delta = 1.2$ pts) while LLM-based methods show meaningful sensitivity ($\Delta = 7.9$ – 10.7 pts), consistent with known mechanisms of bias propagation in generative models.

3 Method

3.1 Embedding-Based Similarity

In Method 2a, we built a fast baseline model called `embedding_matcher.py` that runs locally on the CPU with zero external API costs. The system uses the `all-MiniLM-L6-v2` model from SentenceTransformers to convert text into dense vector representations. Instead of comparing full documents alone, we implemented a section-aware strategy using regular expressions to identify the Skills and Experience sections of each CV and JD. The model computes cosine similarity for each section pair; the final score weights Skills at 40%, Experience at 40%, and overall document similarity at 20%. If a section is not detected, the full document embedding is used as a fallback.

3.2 Structured Extraction and Rule-Based Scoring

Method 2b implements a three-step pipeline across `extraction.py`, `rule_scorer.py`, and `llm_client.py`. First, an LLM call parses raw CV and JD text into a shared `ExtractedProfile` schema containing a normalized skills list, total years of experience, and education details (degree level and field of study). Skill names are lowercased at extraction time and further resolved through an alias dictionary before scoring (e.g. `ml` $\rightarrow$ `machine learning`, `k8s` $\rightarrow$ `kubernetes`). Second, three sub-scores are computed deterministically: skills coverage is the fraction of JD-required skills the candidate matches; experience applies a linear penalty of 25 points per missing year, floored at zero; education checks degree-level hierarchy (BSc < MSc < PhD) and field of study independently, scoring the fraction of required components met. The

final `match_score` is an unweighted average of the available sub-scores. A fourth LLM call scores CV writing clarity on a 0–100 scale independently of qualifications; this is stored as a separate `writing_quality` sub-score and does not enter the `match_score`.

3.3 LLM-as-Judge

Method 2c implements a direct LLM-based scoring approach in `llm_judge_starter.py` using `gemini-3.5-flash-lite` via the OpenAI-compatible Google AI Studio API. The judge receives raw CV and JD text in a single prompt grounded in a two-tier recruiting rubric: must-have requirements (required skills, minimum experience, mandatory education) are evaluated first and penalised more heavily than nice-to-have preferences. Vague, responsibility-only phrasing common in keyword-optimised CVs is explicitly down-weighted relative to concrete, measurable claims. The prompt further instructs the model to handle negation correctly (“no experience with X” is not a skill match), resist hallucination, and credit synonym and implied-experience matches. The model returns a JSON object containing a `match_score` (0–100), sub-scores for skills, experience, and education, and a 2–3 sentence natural-language rationale citing specific matches and gaps. A separate writing-quality signal, sharing its prompt with Method 2b (§3.4), is implemented to rate CV clarity and professionalism on a 0–100 scale independent of qualifications, though it was not computed for the gold-standard runs and is not reported in Table 3.

3.4 Gap Explainer

The Gap Explainer takes the `StructuredDiff` produced by the rule scorer (matched skills, missing skills, experience gap, degree-level match, and field-of-study match) and generates a 2–3 sentence natural-language rationale via an LLM call. The prompt is explicitly grounded in the diff fields so the model cannot introduce information absent from the structured output; if the writing-quality score is notably high (≥ 80) or low (≤ 40), it is appended as a note. A deterministic template fallback fires automatically on any exception, reproducing the same information in fixed phrasing.

Source		Role
Bhawal Dataset (Bhawal, 2021)	Resume	Source CVs across all four job families.
batuhanmtl/job-skill-set (Mutlu, 2024) (Hugging Face)		Source JDs for software/data, HR & admin and sales; stored entirely in upper case.
LinkedIn Postings 2023-2024 (LinkedIn, 2024) (Kaggle)	Job	Source JDs for the logistics family; stored in mixed case.

Table 1: Data sources and their roles.

4 Data and Gold-Standard Labeling

Corpus Construction and Human Annotation

We built the 30-pair gold-standard corpus by sourcing CVs from the Bhawal Resume Dataset (Bhawal, 2021) and JDs from batuhanmtl/job-skill-set (Mutlu, 2024) and LinkedIn’s 2023–2024 postings (LinkedIn, 2024) (for the logistics family), then stratifying pairs across four job families (software/data, HR & admin, sales, logistics) and three fit levels (weak/partial/strong) using per-family TF-IDF cosine similarity thresholds, targeting a roughly even split across all three. Table 1 shows the sources and roles.

We deliberately included multi-column, table-based, and non-native-English CVs for formatting diversity, since robustness to layout is one of the project’s research questions. Two shortcuts were rejected along the way: `argmax-per-resume` sampling, which skewed the corpus toward strong fits, and keyword-based JD classification, which mis-classified postings with non-standard titles; diversity-weighted sampling and a shared category taxonomy were used instead.

Each pair was anonymised into a randomly numbered PDF (PAIR_01 through PAIR_30) with job family and fit level withheld from the filename, to avoid anchoring the annotator. We wrote a methodology report, a labelling rubric with a kappa-based agreement plan (quadratically weighted for the ordinal fields, unweighted for the nominal ones), and a literature/regulatory survey, then completed our own labelling pass across all 30 pairs. The two JD sources use different casing conventions (upper case vs. mixed case); scores were computed on raw text without normalisation, which may introduce minor noise.

Scope	Count
Formatting evaluations per method (12 controls + 48 perturbations)	60
Name/demographic-proxy evaluations per method	108
Total scoring conditions per method	168
Total scoring conditions across all three methods	504
Generated PDFs, all QA-passed against source content	156

Table 2: Robustness testing bundle: scoring conditions and generated PDFs.

Robustness Protocol: Formatting and Demographic-Proxy Sensitivity

Building on that corpus, we designed and built the robustness testing bundle that will run across all three methods, checking formatting sensitivity and demographic-proxy (name) sensitivity independently against a fixed content base. Twelve pairs were selected from the 30 gold-labelled pairs, three per job family, following the actual gold labels rather than a forced grid, since no strong-fit logistics pair was selected for this subset; its anchors are two partial and one weak/reject pair instead. The three source CVs already flagged as multi-column were kept in the subset.

From those 12 pairs we generated 48 formatting perturbations (single-column, two-column, table-based, section-order) and 108 name variants (a neutral control plus eight names spanning White, African-American (Veldanda et al., 2023; Bertrand and Mullainathan, 2004), Turkish (Turkish Statistical Institute (TÜİK), 2025) and Indian (Howell et al., 2025)¹ name groups, one woman-associated and one man-associated name per group), for 156 generated PDFs in total. Every generated PDF passed an automated lexical-content check against its source, and the Turkish and Indian variants were additionally checked by eye given the less common name sets involved. Gold labels carry over unchanged across every variant, since none of the manipulations touch a candidate’s actual qualifications. Table 2 summarizes the full scope of the robustness bundle.

We also defined the analysis metrics in advance (signed and absolute score deltas, decision-flip rate, formatting rank stability, name-group

¹This paper’s stated focus is institutional-prestige bias in peer review rather than a hiring name audit; the Indian-name association should be verified before final submission.

effect, an LLM repeatability check), so scoring can go straight into the run sheets once the other methods are pointed at the bundle. Two caveats worth flagging: the gold labels behind this subset come from a single annotator, and the name proxies mix US race-associated groups (Veldanda et al., 2023; Bertrand and Mullainathan, 2004) with national-cultural groups (Turkish Statistical Institute (TÜİK), 2025; Howell et al., 2025), so results should be read as name-group-specific sensitivity rather than a single unified bias measure.

5 Experimental Setup

5.1 Evaluation Harness

We compare all three methods against the same 30-pair gold-standard dataset using a unified evaluation engine in `eval_harness.py`. The harness normalizes each method’s JSON output into a common format and computes three metrics: **Spearman rank correlation** (ρ) between predicted and human overall-fit scores (both on a 0–100 scale), **shortlisting accuracy** at a threshold of 70 (binary shortlist/reject classification), and **operational efficiency** (average latency per pair in milliseconds, total API token consumption). Human scores are rescaled from the 1–5 annotation scale to 0–100 before all comparisons.

5.2 Bias Robustness Tests

Bias tests run each method’s scoring pipeline on 156 generated PDF variants covering 12 of the 30 gold pairs (three per job family). The 48 formatting variants (single-column, two-column, table-based, section-order) and 108 demographic name variants (a neutral control plus eight names spanning White, African-American, Turkish, and Indian groups) are described in §4. Gold labels carry over unchanged across all variants since no manipulation affects the candidate’s actual qualifications.

6 Results

6.1 Cross-Method Comparison

Table 3 shows a clear accuracy–cost–speed trade-off. Method 2b produces the weakest Spearman correlation ($\rho = 0.177$) despite its four-LLM-call pipeline, indicating that decomposing the match into rule-scored sub-scores does not translate into better rank agreement with human judgement. Method 2a recovers substantial accuracy over 2b at a fraction of the cost (zero API calls, 83 ms per

Method	Spearman ρ	Acc. @70	Avg. Latency	Total Tokens	Avg. Tokens/Pair
2a: Dense Embedding Similarity	0.5169	66.67%	83.23 ms	0	0.0
2b: Structured Extraction + Rules	0.1766	63.33%	4825.33 ms	156,452	5215.1
2c: Direct LLM-as-Judge	0.7882	76.67%	2886.00 ms	88,926	2964.2

Table 3: Evaluation harness comparison across all three methods (n=30).

Method	Format bias		Name bias	
	Avg	Max	Avg	Max
2a	7.4	12.0	1.2	1.9
2b	4.0	16.7	7.9	25.0
2c	12.3	26.0	10.7	27.0

Table 4: Score range (pts) across layout and name variants (n=12 pairs). Lower is more robust.

pair), though its 66.7 % shortlisting accuracy exactly matches the majority-class baseline of always predicting “reject” (20 of 30 gold pairs), so its gain over a naive baseline shows up in ranking quality rather than binary classification. Method 2c achieves the best overall performance with the highest Spearman correlation and shortlisting accuracy, while using fewer tokens per pair than Method 2b because it makes a single end-to-end call rather than four. Figure 1 shows the predicted–human score distributions across all three methods.

6.2 Bias Robustness

Table 4 and Figure 2 report the score range across formatting and demographic name variants for each method. For format bias, Method 2b is the most robust (avg 4.0 pts) because its intermediate structured representation absorbs minor layout noise during extraction; Method 2c is the most sensitive (avg 12.3 pts, max 26 on PAIR_27), consistent with the LLM processing raw text where layout affects content presentation directly. For name bias, Method 2a shows near-zero sensitivity (avg 1.2 pts) as expected. It is obvious that dense embeddings do not encode name identity. Method 2b shows higher name sensitivity than anticipated (avg 7.9 pts, max 25 on PAIR_15 and PAIR_25), suggesting that the LLM extraction step picks up demographic name cues even though names are irrelevant to skills, experience, and education. Method 2c shows the highest name sensitivity (avg 10.7 pts, max 27 on PAIR_10), consistent with the model processing candidate names as part of its holistic fit assessment.

7 Discussion

Accuracy–cost trade-off. The results confirm a clear gap between methods, but not the hierarchy we had originally expected. Method 2a’s Spearman correlation ($\rho = 0.517$) is higher than anticipated, though it still falls short of Method 2c’s. The scores are internally consistent, but cosine similarity measures something fundamentally different from what recruiters weight. Embedding similarity rewards topical overlap, while human annotators penalise missing must-have constraints regardless of topic proximity. A candidate with strong general data-science vocabulary but no required certification can still score reasonably well under Method 2a and low under human judgment. Method 2b, despite decomposing the match into auditable sub-scores, produces the weakest correlation of the three methods ($\rho = 0.177$), and its four-LLM-call pipeline costs nearly twice the tokens of Method 2c (5,215 vs. 2,964 per pair) while achieving lower accuracy than the free Method 2a as well. Part of this gap traces to a rule-scorer implementation issue documented in §8: when JD skill extraction returns no required skills, the scorer currently defaults the skills sub-score to a perfect 100 rather than treating it as missing, which manufactures spuriously high match scores for a handful of pairs. Method 2c’s single end-to-end call is not only more accurate ($\rho = 0.788$, 76.7% shortlisting accuracy) but also more token-efficient than Method 2b, suggesting that holistic LLM reasoning on raw text suits the task better than decomposed rule application.

Structured representations absorb layout noise but not name cues.

The bias results contain a finding that was not anticipated at the design stage. Method 2b was expected to be robust to demographic name cues, since names are irrelevant to the three fields it scores: skills, experience, and education. In practice, its name sensitivity (avg range 7.9 pts, max 25 pts on PAIR_15 and PAIR_25) is comparable to Method 2c (avg 10.7 pts). The most likely explanation is that the LLM extraction step processes the full CV text and encodes name-

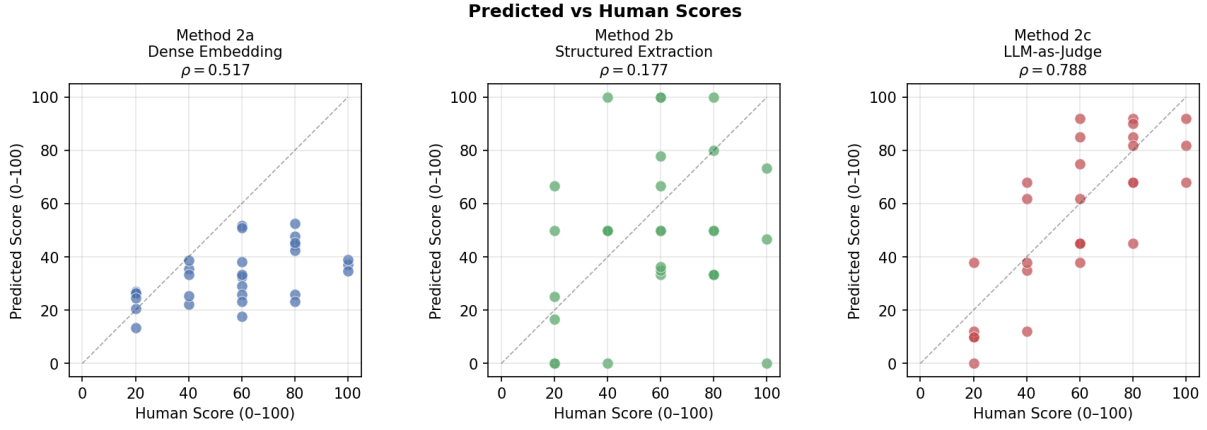


Figure 1: Predicted vs. human scores for all three methods ($n=30$). The dashed line marks perfect agreement. Method 2b shows no systematic trend; Method 2a shows a moderate positive trend; Method 2c clusters most tightly around the diagonal.

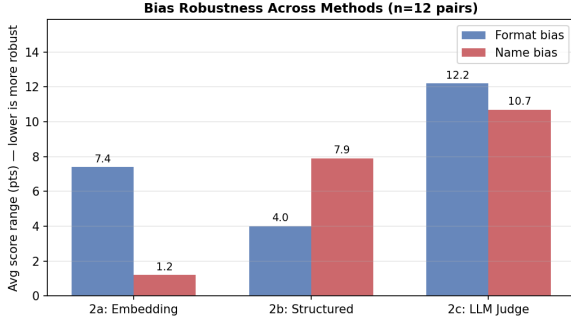


Figure 2: Average score range (pts) across formatting and name variants per method ($n=12$ pairs). Lower values indicate greater robustness.

associated priors before passing the structured output to the deterministic scorer. In other words, bias enters at the extraction stage, not the scoring stage. Format bias behaves as expected: Method 2b is the most robust (avg 4.0 pts) because its intermediate schema absorbs layout differences, while Method 2c is the most sensitive (avg 12.3 pts) because the raw text passed to the judge changes with layout.

Practical implications. For deployment scenarios with hard latency and cost constraints, Method 2a remains a viable first-pass filter. It runs in under 100 ms with zero API cost and now shows a moderate correlation with human judgment ($\rho = 0.517$), which is sufficient to reduce the candidate pool before a more expensive method runs, even though its 66.7% shortlisting accuracy only matches the majority-class baseline. Method 2c is better suited to final shortlisting, where ranking quality and explainability matter. Method 2b’s structured diff and gap explanation still provide the audit trail

required in regulated hiring contexts (Raghavan et al., 2020), but its current ranking quality trails both other methods, so it is best positioned as an auditability layer rather than as a source of ranking improvement over Method 2a. Name bias in both LLM-based methods warrants attention before production deployment. Even if individual score shifts are modest, systematic shifts correlated with demographic proxies can produce disparate shortlisting rates at scale.

8 Limitations

8.1 Precision on borderline cases

Method 2c’s main weakness is separating *shortlist* from *borderline*: it tends to score strong-but-limited candidates (e.g. overqualification, one missing nice-to-have) about as high as clean shortlists. This mirrors a defensible real-world early-pass posture (favour recall, let a human make the final call) but is worth stating as a limitation rather than a strength.

8.2 Single annotator

The 30-pair gold standard was labelled by one annotator. A second annotator was recruited for the IAA subset but results were not available before submission. All correlation and accuracy figures should therefore be interpreted as agreement with one human judgement, not with a consensus label. Quadratically weighted κ for the ordinal fields and unweighted κ for nominal ones remain to be computed.

8.3 Rate-limit-induced latency

Several calls during full 30-pair evaluation runs took 25–70 seconds (vs. a typical 1–2 s), occurring immediately before free-tier quota limits were reached. Reported latency figures represent median behaviour; tail latency under quota pressure is substantially higher.

8.4 Writing-quality leakage into match score

On a synthetic poorly-written-but-qualified CV, `match_score` dropped modestly (86→82) relative to a clean version of the same content, attributed partly to lack of concrete measurable impact metrics. The writing-quality and match signals are designed to be independent; this result suggests minor leakage worth monitoring in production use.

8.5 Skills sub-score default when JD skill extraction is empty

In `rule_scorer.py`, `skills_score` defaults to a perfect 100 whenever JD skill extraction returns zero required skills, instead of being treated as missing information the way the experience and education sub-scores are. This affects 4 of the 30 gold pairs (PAIR_06, PAIR_09, PAIR_10, PAIR_26); for two of them (PAIR_06, PAIR_09) skills was the *only* available sub-score, so the default becomes the entire `match_score`, producing spuriously “perfect-fit” scores of 100 against human labels of 60–80. We simulated the seemingly correct fix – excluding the skills sub-score when JD extraction is empty – and found it makes Method 2b’s overall correlation *worse* (from $\rho = 0.177$ to $\rho = 0.073$), because it exposes a second, compounding issue: `match_score` falls back to 0.0 when no sub-scores are available at all, turning “no information” into a hard fail rather than an excluded or neutral prediction. We report Method 2b’s results under the current implementation and flag both issues as an open design problem for the rule scorer’s missing-data handling, rather than silently trading one default-value bug for another.

8.6 Education is the weakest-correlated sub-dimension

Education sub-scores (Method 2c) correlate with human judgements at $\rho = 0.70$, below skills ($\rho = 0.84$) and experience ($\rho = 0.85$). A recurring driver is degree prestige: on PAIR_01, the judge scored education 90 citing an advanced Mechanical Engineering degree, while the human rated it

20 because the role required warehouse operations management with no engineering component.

9 Conclusion

We presented ECMS, a three-method system for explainable CV–job matching evaluated against a 30-pair gold-standard dataset with human annotations. The methods span a clear accuracy–cost–speed spectrum: dense embedding similarity (2a) provides instant, zero-cost filtering with a moderate correlation to human judgment ($\rho = 0.517$), though it still misses some of the hard recruitment constraints that drive human fit judgements; structured extraction with rule-based scoring (2b) recovers interpretability, but not accuracy ($\rho = 0.177$, the lowest of the three methods), through an auditable structured diff and natural-language gap explanation; and direct LLM-as-judge evaluation (2c) achieves the strongest alignment with human judgment ($\rho = 0.788$, 76.7% shortlisting accuracy) with fewer tokens per pair than the pipeline-based approach.

Bias robustness tests across 48 formatting and 108 demographic name variants reveal that surface robustness varies by method design: embedding-based scoring is nearly immune to name swaps (avg range 1.2 pts) but moderately sensitive to layout; structured extraction absorbs layout noise well (avg range 4.0 pts) but shows unexpected name sensitivity at the extraction stage; and the LLM judge is the most sensitive to both perturbations. Taken together, the results suggest that no single method dominates across all three desiderata of accuracy, explainability, and robustness — a practical deployment would chain Method 2a as a fast first-pass filter with Method 2c for final shortlisting, while Method 2b offers the strongest audit trail for regulated hiring contexts. Future work should address inter-annotator agreement on the gold standard, larger-scale evaluation across more job families, and mitigation strategies for name-associated bias in LLM extraction pipelines.

References

- Marianne Bertrand and Sendhil Mullainathan. 2004. [Are Emily and Greg more employable than Lakisha and Jamal? a field experiment on labor market discrimination](#). *American Economic Review*, 94(4).
- Snehaan Bhawal. 2021. [Resume dataset](#). Kaggle.
- Tolga Bolukbasi, Kai-Wei Chang, James Zou,

-
- Venkatesh Saligrama, and Adam Kalai. 2016. Man is to computer programmer as woman is to homemaker? Debiasing word embeddings. In *Advances in Neural Information Processing Systems*, volume 29.
- Maria De-Arteaga, Alexey Romanov, Hanna Wallach, et al. 2019. Bias in Bios: A case study of semantic representation bias in a high-stakes setting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency*, pages 120–128.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT 2019*, pages 4171–4186.
- Anthony Howell, Jieshu Wang, Luyu Du, Julia Melkers, and Varshil Shah. 2025. [Prestige over merit: An adapted audit of LLM bias in peer review.](#)
- LinkedIn. 2024. [Job postings \(2023–2024\).](#) Kaggle.
- Batuhan Mutlu. 2024. [Job skill set dataset.](#) Hugging Face.
- Chen Qin, Hengshu Zhu, Tong Xu, Chen Zhu, Liang Jiang, Enhong Chen, and Hui Xiong. 2018. Enhancing person-job fit for talent recruitment: An ability-aware neural network approach. In *Proceedings of the 41st International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 25–34.
- Manish Raghavan, Solon Barocas, Jon Kleinberg, and Karen Levy. 2020. [Mitigating bias in algorithmic hiring: Evaluating claims and practices.](#) In *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency*, pages 469–481. ACM.
- Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, pages 3982–3992. ACL.
- Turkish Statistical Institute (TÜİK). 2025. Most used given names; most used surnames. Given names: <https://nip.tuik.gov.tr/?value=EnCokKullanilanIsimler>. Surnames: <https://nip.tuik.gov.tr/?value=EnCokKullanilanSoyIsimler>.
- Akshaj Kumar Veldanda, Fabian Grob, Shailja Thakur, Hammond Pearce, Benjamin Tan, Ramesh Karri, and Siddharth Garg. 2023. [Are Emily and Greg still more employable than Lakisha and Jamal? investigating algorithmic hiring bias in the era of ChatGPT.](#)
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, et al. 2023. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *Advances in Neural Information Processing Systems*, volume 36.